TIỂU LUẬN MÔN HỌC

HỌC MÁY / KHAI PHÁ DỮ LIỆU

CÀI ĐẶT VÀ ĐÁNH GIÁ LOGISTIC REGRESSION

Tự cài đặt bằng NumPy và so sánh với scikit-learn

Dữ liệu: Breast Cancer Wisconsin (Diagnostic) (UCI ML Repository)

Sản phẩm nộp: Jupyter notebook logistic_regression.ipynb

Ngày: 28/09/2026

MỤC LỤC

1. 1. Giới thiệu và mục tiêu
2. 2. Dữ liệu và tỷ lệ các lớp
3. 3. Phương pháp: cài đặt Logistic Regression bằng NumPy
4. 4. Kết quả thực nghiệm
5. 5. Nhận xét
6. 6. Phụ lục - Hướng dẫn chạy

1. Giới thiệu và mục tiêu

Bài tiểu luận tự cài đặt mô hình Logistic Regression (hồi quy logistic) cho bài toán phân loại nhị phân bằng NumPy — không dùng mô hình hay bộ tối ưu có sẵn — sau đó so sánh với sklearn.linear_model.LogisticRegression trên cùng dữ liệu đã tiền xử lý. Mục tiêu:

- Chuẩn bị dữ liệu: mô tả đặc trưng, nhãn và tỷ lệ các lớp.
- Cài đặt bằng NumPy: score, sigmoid, log loss và cập nhật trọng số + bias bằng gradient descent.
- Huấn luyện sklearn LogisticRegression trên cùng dữ liệu.
- Đánh giá: accuracy, precision, recall, F1, ROC-AUC, log loss + ma trận nhầm lẫn của mỗi bản.
- Nhận xét về hội tụ, chênh lệch giữa hai bản và các lỗi phân loại thường gặp.

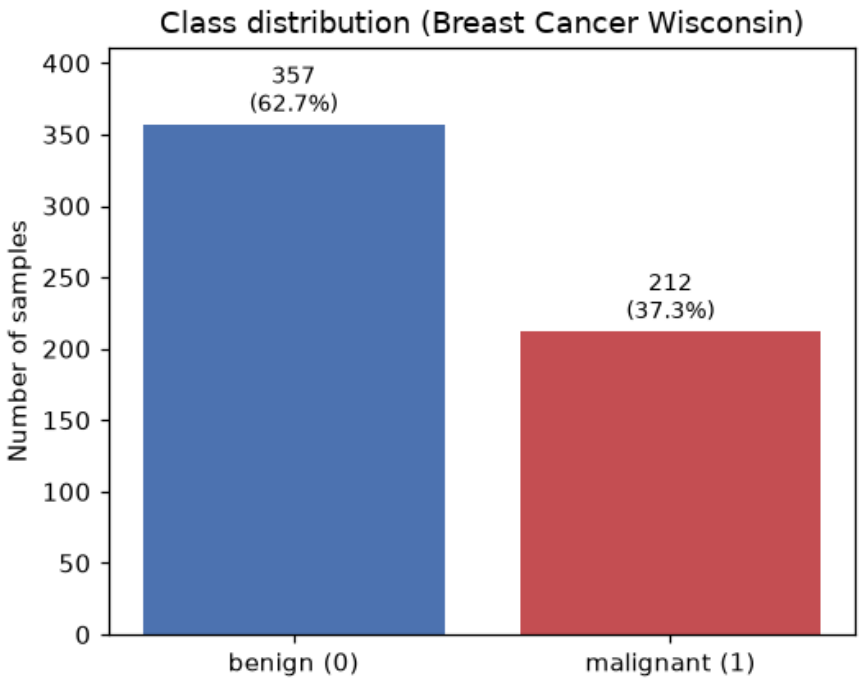
2. Dữ liệu và tỷ lệ các lớp

Bộ dữ liệu Breast Cancer Wisconsin (Diagnostic) gồm 569 mẫu, 30 đặc trưng số thực (mô tả đặc điểm nhân tế bào từ ảnh sinh thiết: bán kính, kết cấu, chu vi, diện tích, độ lõm...). Đây là dữ liệu y tế thực tế từ UCI, nạp offline qua sklearn nên notebook chạy được ở mọi máy, không cần tải dữ liệu ngoài.

Nhãn là nhị phân. Ta quy ước lớp dương (y=1) là 'malignant' (ác tính) — lớp cần phát hiện trong y khoa. Tỷ lệ các lớp:

- Ác tính (malignant, y=1): 212 mẫu (37.3%).
- Lành tính (benign, y=0): 357 mẫu (62.7%).

Dữ liệu được chia train/test = 75%/25% có phân tầng (stratify) để giữ tỷ lệ lớp (426 train / 143 test), và chuẩn hóa bằng StandardScaler (fit trên tập train) để tránh rò rỉ dữ liệu. Cùng dữ liệu này dùng cho cả hai bản.



Hình 1. Phân bố các lớp trong bộ dữ liệu.

3. Phương pháp: cài đặt Logistic Regression bằng NumPy

Với vector đặc trưng x, trọng số w và bias b:

- Score tuyến tính: $z = x \cdot w + b$.
- Sigmoid (xác suất): $p = \sigma(z) = 1 / (1 + e^{(-z)})$; clip z vào $[-500, 500]$ để tránh tràn số.
- Log loss (binary cross-entropy): $L = -(1/m)\ \Sigma\ [y \cdot \log(p) + (1-y) \cdot \log(1-p)]$; clip p vào $[\varepsilon, 1-\varepsilon]$ để tránh log(0).
- Điều chuẩn L2 (chỉ trên trọng số): $J = L + (\lambda/2) \cdot \|w\|^2$. Do dữ liệu gần tách tuyến tính, nếu không điều chuẩn thì trọng số sẽ tiến ra vô cực và xác suất bão hòa (log loss test rất lớn); L2 giữ nghiệm ổn định.
- Gradient: $dJ/dw = (1/m) \cdot X^T(p-y) + \lambda \cdot w$; $dJ/db = (1/m) \cdot \Sigma(p-y)$.
- Cập nhật (gradient descent): $w \leftarrow w - \eta \cdot dJ/dw$; $b \leftarrow b - \eta \cdot dJ/db$.

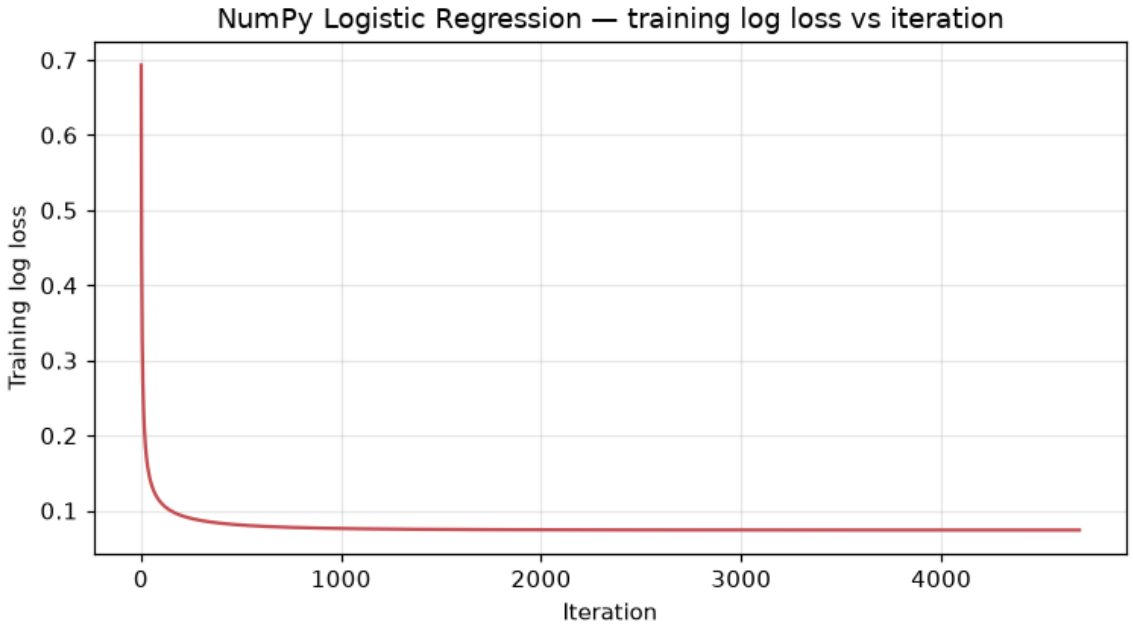
Siêu tham số: learning rate $\eta = 0.1$, tối đa 5000 vòng lặp, điều kiện dừng khi |thay đổi log loss| < 1e-08, L2 $\lambda = 0.01$. Khởi tạo $w = 0$, $b = 0$.

Để so sánh công bằng, sklearn dùng cùng mức điều chuẩn: sklearn tối ưu $C \cdot \Sigma BCE + \frac{1}{2}\|w\|^2$, còn bản NumPy tối ưu $(1/m)\Sigma BCE + (\lambda/2)\|w\|^2$; khớp hai gradient cho $C = 1/(m \cdot \lambda)$. Nhờ đó cả hai giải cùng một hàm mục tiêu.

4. Kết quả thực nghiệm

4.1. Sự hội tụ của gradient descent

Vì đặc trưng đã chuẩn hóa và hàm log loss là lồi, gradient descent giảm đơn điệu và hội tụ mượt. Quá trình dừng sau 4690 vòng (đã hội tụ), log loss train cuối = 0.0741.



Hình 2. Log loss trên tập train theo vòng lặp (đường hội tụ).

4.2. Bảng so sánh các chỉ số

Bảng 1. So sánh trên tập test (ROC-AUC và log loss tính từ xác suất dự đoán).

Chỉ số	NumPy (tự cài)	scikit-learn
Accuracy (độ chính xác)	0.9790	0.9790
Precision (độ chuẩn xác)	1.0000	1.0000
Recall (độ bao phủ)	0.9434	0.9434
F1-score	0.9709	0.9709
ROC-AUC	0.9983	0.9983
Log loss	0.0753	0.0753

Hai bản cho kết quả gần như trùng khớp: accuracy = 0.9790, ROC-AUC = 0.9983, log loss = 0.0753. Precision = 1.00 và Recall = 0.9434 cho lớp ác tính.

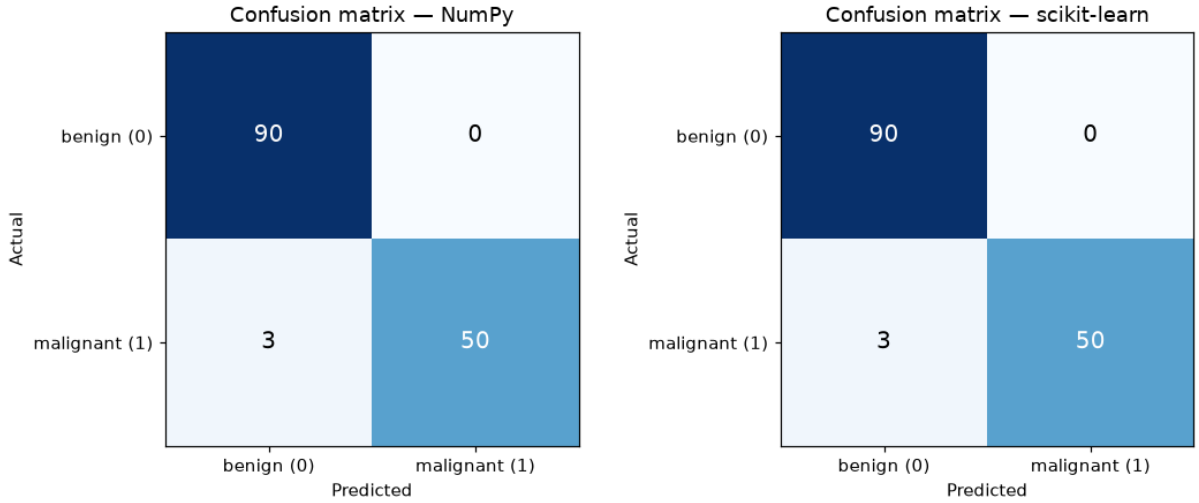
4.3. Ma trận nhầm lẫn

Bản NumPy:

	Dự đoán: benign	Dự đoán: malignant
Thực tế: benign	90	0
Thực tế: malignant	3	50

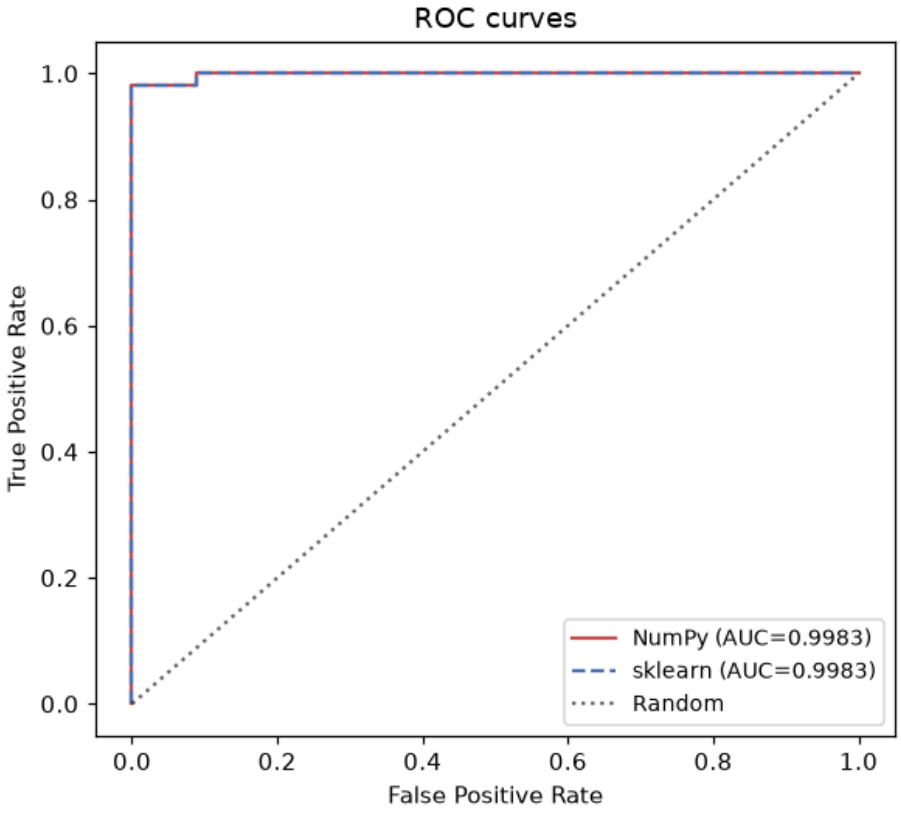
Bản scikit-learn:

	Dự đoán: benign	Dự đoán: malignant
Thực tế: benign	90	0
Thực tế: malignant	3	50



Hình 3. Ma trận nhầm lẫn của hai bản.

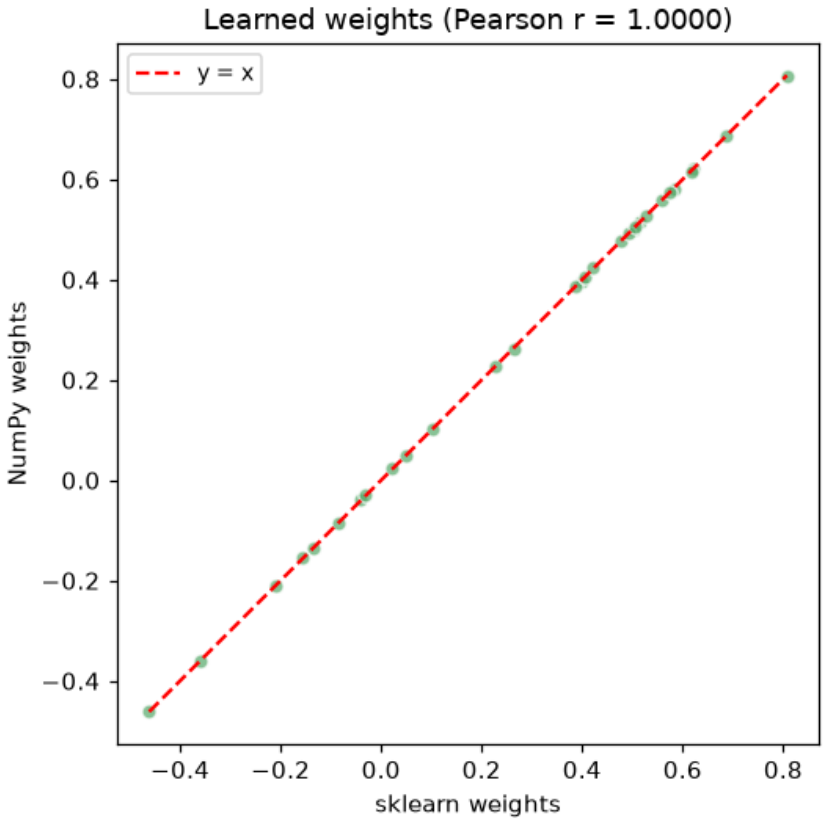
4.4. Đường cong ROC



Hình 4. Đường ROC (AUC NumPy = 0.9983, sklearn = 0.9983).

4.5. So sánh trọng số học được

Hệ số tương quan Pearson giữa trọng số hai bản = 1.0000; bias NumPy = -0.464, bias sklearn = -0.464. Trọng số gần như trùng khớp — minh chứng bản NumPy cài đúng.



Hình 5. So sánh trọng số NumPy vs sklearn.

5. Nhận xét

Về sự hội tụ:

Chuẩn hóa đặc trưng giúp gradient descent hội tụ nhanh và ổn định trên mặt log loss lồi; đường log loss (Hình 2) phẳng dần và dừng theo ngưỡng 1e-08 sau 4690 vòng.

Về chênh lệch giữa hai bản:

Do cùng tối ưu một hàm mục tiêu (log loss + L2 khớp nhau), hai bản hội tụ về gần như cùng nghiệm: trọng số tương quan ≈ 1.00, bias trùng nhau, và cả 6 chỉ số khớp tới 4 chữ số thập phân — dù dùng bộ tối ưu khác nhau (gradient descent bước cố định so với lbfgs kiểu tựa-Newton). Với hàm lồi, bộ tối ưu chỉ ảnh hưởng đường đi, không ảnh hưởng đích đến.

Về các lỗi phân loại thường gặp:

Trên 143 mẫu test, bản NumPy sai 3 mẫu, sklearn sai 3 mẫu, trong đó 3 mẫu bị sai ở cả hai bản. Đây là các ca ranh giới có xác suất dự đoán gần 0.5 — tức mẫu thực sự nhập nhằng, không phải

lỗi của cách cài đặt. Trong y khoa, có thể hạ ngưỡng quyết định để tăng recall (giảm bỏ sót ca ác tính) đổi lấy giảm precision.

6. Phụ lục - Hướng dẫn chạy

Sản phẩm nộp chính: bai-tap-2/logistic_regression.ipynb (chạy được từ đầu đến cuối).

Các bước chạy (Windows PowerShell):

```
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
jupyter notebook bai-tap-2\logistic_regression.ipynb
```